ELL305 Computer Architecture

Term Paper — Draft #1 (v0.1)

Progress Report: Work Done as of 24 August 2026

[Your Name], [Entry Number]
[Partner Name], [Entry Number]

24 August 2026

1 Topic, Scope and Supervision

1.1 Chosen Topic

The chosen topic is **memory consistency models** in shared-memory multiprocessors.

Working title.
Revisiting Sequential Consistency: Memory Consistency Models, Their Cost, and Whether Speculation Has Made Strong Ordering Affordable

Scope. A shared-memory multiprocessor must specify when, and in what order, one core's writes become visible to the others. That specification is the machine's memory consistency model, and it is an architectural contract as binding as the instruction set. The paper covers the design space from sequential consistency through TSO, PSO, weak ordering and release consistency; the mechanisms that make relaxation necessary (store buffers, out-of-order execution) and those that could make it unnecessary (speculative execution with coherence-based detection); and the historical argument that led the industry to reject sequential consistency around 1990.

Guiding question. Sequential consistency was rejected as a hardware memory model on performance grounds in the late 1980s, on machines that did not speculate. A modern out-of-order core already contains both mechanisms a strong model requires — coherence invalidations that announce remote writes, and a squash-and-restart path built for branch misprediction recovery. The paper asks whether the performance objection still holds, and, if it does not, what else accounts for the continued absence of sequentially consistent hardware.

The work is organised around four questions:

Q1 Formal
: Can SC, TSO, PSO, weak ordering and release consistency be expressed as successively weaker constraints on a single set of ordering relations, with strict containment proved rather than asserted?

Q2 Empirical
: Which relaxations are *observable* on real x86 and ARM hardware, as distinct from those merely *permitted* by the architecture?

Q3 Quantitative
: Holding the microarchitecture fixed and varying only the memory model, what does sequential consistency cost with and without speculative load execution, as core count rises?

Q4 Interpretive
: If speculative SC proves inexpensive, what accounts for its absence from shipping processors?

1.2 Mapping to the Prescribed Textbook

The prescribed text is S. R. Sarangi, *Basic Computer Architecture*, version 3.09 [26]. The topic maps principally to **Chapter 12, *Multiprocessor Systems***, and within it to **§12.4, *MIMD Multiprocessors***.

§	Title (page)	Role in the paper
Primary — Chapter 12		
12.4.1	Logical Point of View (579)	Shared-memory abstraction.
12.4.2	Coherence (581)	The coherence/consistency distinction, developed at length.
12.4.3	**Memory Consistency (583)**	**The core section. The paper is an extension of this.**
12.4.5	Shared Caches (596)	Background.
12.4.6	Coherent Private Caches (596)	Extended to full MESI/MOESI and directory protocols.
12.4.7	Implementing a Memory Consistency Model* (603)	Direct antecedent of the speculative-SC discussion.
Supporting		
10.4	Pipeline Hazards (420)	Reordering and dependences.
10.9	Performance Metrics (463)	Benchmarking framework.
10.11.4	Out-of-Order Pipelines (492)	Speculation and the squash mechanism.
11.2	Caches (517)	Private caches, non-blocking misses.
11.3	The Memory System (532)	Latency figures motivating the store buffer.
12.2.2	Shared Memory vs Message Passing (571)	Framing.
12.2.3	Amdahl's Law (576)	Scaling discussion.
12.6	Interconnection Networks (620)	Coherence latency in the simulator.
App. A	Cortex-A8/A15, Sandy Bridge (734–746)	Real-machine case studies.

Table 1: Mapping of the topic onto the prescribed text. * denotes a section marked advanced.

Additional texts. Because §12.4.3 treats only sequential consistency and the relaxed models are outside the scope of the prescribed text, three further sources carry the technical weight of the paper. All are fully specified, with reproduced tables of contents, in Chapter 7 of the paper draft.

- S. R. Sarangi, *Next-Gen Computer Architecture: Till the End of Silicon*, White Falcon 2023, ISBN 8119510143 [25]. Chapter 9, *Multicore Systems: Coherence, Consistency and Transactional Memory*, runs to roughly 118 pages and is the source of the formalism used throughout.
- V. Nagarajan, D. J. Sorin, M. D. Hill and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd ed., Springer 2020 [23]. The standard reference on the subject.
- Vendor architecture manuals (Intel SDM Vol. 3A; Arm ARM Ch. B2; RISC-V RVWMO), cited in preference to any textbook wherever the paper states what a particular instruction set guarantees.

1.3 Advisor

I will consult **Prof. [Sumantra Dutta Roy / Subrat Kar — SELECT ONE]** as advised.

Zotero library `ELL305-MemoryConsistency` has been created and will be shared with Prof. Subrat Kar. Bibliography is exported to `refs.bib` and rendered via BibTeX.

2 Technical Foundations Established

This section records the material understood so far, in our own words. A statement of comprehension is stronger evidence of progress than a percentage, and the argument of the full paper depends on distinctions that are easy to state incorrectly.

2.1 Why Shared Memory Became Unavoidable

For roughly the first four decades of commercial computing, programs were made faster by making one processor faster: raising the clock, and doing more work per cycle by overlapping instruction execution. The course's pipelining material (Sarangi §10.4 on hazards, §10.7 on forwarding) and, historically, Kogge [18] are precisely the story of that overlap. Sarangi's advanced volume continues the arc into out-of-order pipelines, where instructions are deliberately executed in an order different from the program's, subject only to true data and control dependences [25].

Two physical limits ended this. Frequency scaling stalled because power dissipation grows steeply with clock speed, and the returns from making a single core wider diminished. After roughly 2005–2010 single-core performance saturated, and the transistors Moore's law kept delivering were redirected from making one core more complex to placing *many* cores on one chip. Hennessy and Patterson frame the same inflection as the turn to thread-level parallelism [14]. The multicore era was therefore not a design preference but a consequence.

Multicore hardware only delivers performance if software can use many cores at once, and the dominant model is **shared memory**: several cores read and write one common address space, communicating implicitly through loads and stores rather than explicit messages (§12.2.2). It is attractive because it extends the uniprocessor programming model — the same pointers, arrays and variables. That convenience conceals the problem this paper is about.

2.2 The Uniprocessor Contract, and Three Ingredients of Reordering

A programmer can reason about a sequential program only because the hardware works hard to provide one guarantee: **the appearance of program order**. A modern pipeline has many instructions in flight and an out-of-order core may complete them in a scrambled sequence, but the architecture ensures the observable result is as if instructions executed one at a time in program order. The commit stage restores program order, and the load–store queue enforces the necessary memory-ordering constraints.

The key point: on a uniprocessor, reordering is *invisible*. This is why a programmer of a sequential program need never learn what a store buffer is.

Three mechanisms reorder or delay memory operations. Each is individually benign on one core, and each becomes externally observable through the eyes of *another* core.

1. **Out-of-order execution.** A core issues independent instructions as operands become ready. Two loads to different addresses may execute in either order; a load may execute before an older store to a different address (§10.11.4).

2. **Store buffers.** To avoid stalling on the latency of making a write globally visible, a core places it in a private FIFO queue and continues. Adve and Gharachorloo [2] use exactly this as their first example of how sequential consistency is violated.

3. **Caches and the interconnect.** Writes propagate to other cores only through the coherence mechanism, and different cores may learn of independent writes in different orders unless the system prevents it (§11.2, §12.4.6).

2.3 The Store Buffer in Detail

The store buffer deserves particular attention, because it causes the behaviour that defines the field.

Access to main memory costs on the order of 200 cycles against roughly one for arithmetic (§11.1, §11.3). The store buffer works because of an asymmetry: a load produces a value subsequent instructions need, so the core must wait; a store produces nothing the core needs — the value is already in hand — so no dependence forces a wait. The store goes into a queue of typically 40 to 60 entries and the core proceeds.

The cost avoided is larger than it appears. Writing to a line requires holding it in a state permitting modification; if the core does not, it must request exclusive ownership and wait for every other holder to relinquish its copy, potentially hundreds of cycles.

A load from the same core checks the store buffer before the cache and is served from any matching pending entry — *store-to-load forwarding*. This is why single-threaded programs are unaffected: a core always observes its own writes in program order, whatever the rest of the system sees.

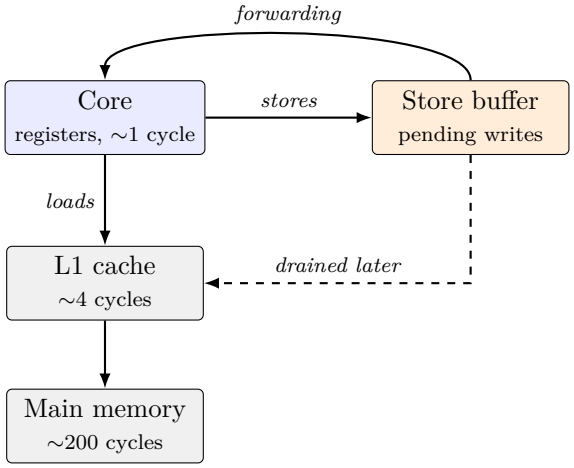

Figure 1: The memory path of a single core. Stores bypass the wait; the store buffer drains asynchronously.

The essential point: *the store buffer is fast precisely because a store held in it is not yet visible to other cores.* The invisibility is not a defect better engineering could remove. It is the same fact as the performance benefit, seen from the other side.

2.4 Coherence Is Not Consistency

Before going further we must fix a distinction the course texts are careful about and that beginners routinely blur (§12.4.2–§12.4.3).

Cache coherence concerns a *single* memory location. It guarantees that all cores agree on one total order of the writes to any one location, and that a read eventually sees the latest write to it. This is what MSI, MESI and MOESI provide (§12.4.6). It is a per-address property and says nothing about how accesses to *different* addresses are ordered. Equivalently, coherence is *per-location sequential consistency*.

Memory consistency concerns ordering *across* locations. If a core writes `x` and then writes `y`, can another core see the new `y` but the old `x`? Coherence has nothing to say, because they are different locations.

A useful way to hold the distinction: *coherence makes each variable behave sanely on its own; consistency governs how variables behave with respect to each other.*

There is also a structural reason coherence cannot help, worth stating plainly: *the store buffer sits between the core and the cache, and therefore upstream of the coherence protocol entirely.* A store resting in the buffer has not been presented to the coherence mechanism. There is nothing for the protocol to order, invalidate or broadcast.

	Coherence	**Consistency**
Scope	One location	Across locations
Guarantee	One total order of writes per location; eventual propagation	Which orderings of all memory operations are permitted
Mechanism	MSI / MESI / MOESI	Store-buffer and pipeline ordering rules; fences
Acts at	The cache	Between core and cache
Course ref.	§12.4.2, §12.4.6	§12.4.3, §12.4.7

Table 2: Coherence versus consistency at a glance. The fourth row is why a perfectly coherent machine still exhibits the anomaly below.

2.5 Litmus Tests: The Vocabulary of the Field

The field's working vocabulary is a small family of tiny two- and four-thread programs called **litmus tests**. Each isolates one ordering question, and each memory model is characterised by which outcomes it permits. These recur throughout the paper and are the basis of Experiment A.

Store Buffering (SB). Initially `x = y = 0`.

Core 1	**Core 2**
`x = 1`	`y = 1`
`r1 = y`	`r2 = x`

Can `r1 = 0` and `r2 = 0`? Intuitively no: imagining the four operations interleaved in some single global order, by the time both reads have happened both writes have occurred, so at least one read should see 1. Yet on essentially every mainstream processor, including an ordinary x86 laptop, both reads can return zero in the same execution. Nothing is broken. Each core holds its write in a private store buffer and proceeds to its read before that write has drained; both reads slip in front of both writes.

Under sequential consistency this is forbidden. Under TSO it is permitted — it is the *defining* relaxation of TSO [27]. This tiny program is the entire memory-consistency problem in miniature, and it is the core of Dekker's algorithm: on hardware permitting this outcome, Dekker's mutual exclusion admits two threads into the critical section at once.

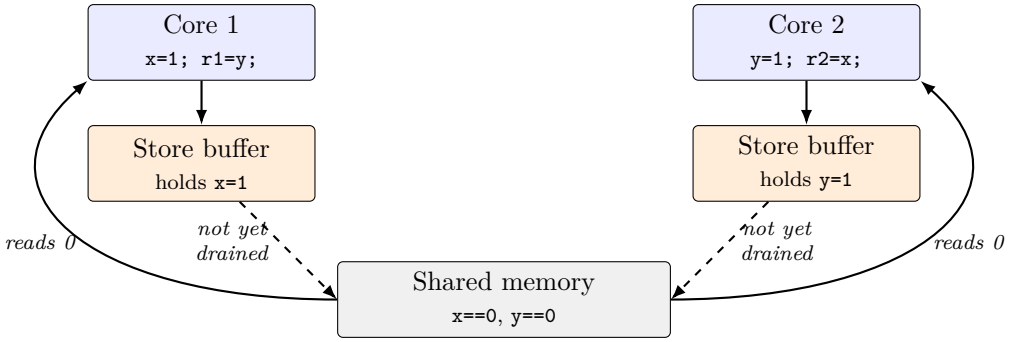

Figure 2: Store buffering, mechanistically. This is the operational (abstract-machine) explanation that Sewell et al. make precise for x86-TSO [27].

Message Passing (MP). Initially `data = 0`, `flag = 0`.

Core 1 (producer)	**Core 2 (consumer)**
`data = 42`	`while (flag == 0) {}`
`flag = 1`	`r = data`

When Core 2 sees `flag == 1`, is `r == 42` guaranteed? Under SC, and for this pattern under TSO (which preserves store→store order), yes. Under weakly ordered models — ARM, POWER, Alpha — **not without a barrier**: the producer's two writes can become visible out of order, or the consumer's two reads can be reordered, so the consumer sees the flag set and reads stale data. This is the single most important pattern in real concurrent code and the reason acquire/release fences exist [21].

Load Buffering (LB). Initially `x = y = 0`. Core 1 reads `x` then writes `y`; Core 2 reads `y` then writes `x`. Can both reads return 1? This requires each core's read to be reordered after its write. Forbidden by SC and TSO; allowed by some weak models. It distinguishes models that preserve load→store order from those that do not.

Independent Reads of Independent Writes (IRIW). Two cores write; two others read both variables in opposite orders. Can the two observers disagree about the order in which the two independent writes occurred? Models permitting this are **not multicopy atomic** — a write can become visible to some cores before others. Classic POWER and ARMv7 permit it; x86-TSO does not, and ARMv8 was revised in 2018 to forbid it. This is the cleanest probe of write atomicity, one of the deepest axes in the subject [4].

Test	**SC**	**x86-TSO**	**SPARC PSO**	**ARMv7 / POWER**	**ARMv8**
SB	Forbidden	Allowed	Allowed	Allowed	Allowed
MP	Forbidden	Forbidden	Allowed	Allowed	Allowed
LB	Forbidden	Forbidden	Forbidden	Allowed	Allowed
IRIW	Forbidden	Forbidden	Forbidden	Allowed	Forbidden

Table 3: Canonical litmus tests and whether each model permits the surprising outcome. To be confirmed empirically in Experiment A and cross-checked against `herd7` predictions.

2.6 The Design Space of Hardware Models

A **memory consistency model** is the architecturally visible specification of which orderings a multiprocessor may exhibit, and therefore which values a read may return. It is a contract: hardware promises no less, software may assume no more.

Models are characterised by which of four program-order pairs they preserve, and by whether they are multicopy atomic.

Model	S→L	S→S	L→L	L→S	MCA	Deployed in
Sequential consistency	✓	✓	✓	✓	✓	—
TSO	×	✓	✓	✓	✓	x86, SPARC
PSO	×	×	✓	✓	✓	SPARC
Weak / RMO	×	×	×	×	varies	ARM, POWER, RISC-V
Release consistency	×	×	×	×	varies	C++11, ARMv8

Table 4: Program-order pairs preserved by each model; MCA denotes multicopy atomicity. Every commercially deployed instruction set sits strictly below sequential consistency.

The tension is visible in the table. At the top, intuition is sound and the programmer needs no special knowledge — but naively the core must stall on every store, abandoning the store buffer and much of its performance. At the bottom the hardware runs at full speed and the programmer is handed fences. Empirically, programmers place them wrongly; the resulting defects are probabilistic, appear only under particular thread placements, and typically vanish under a debugger.

2.7 Historical Development

The industry did not arrive at weak models by oversight. It argued for them, under assumptions that have since changed.

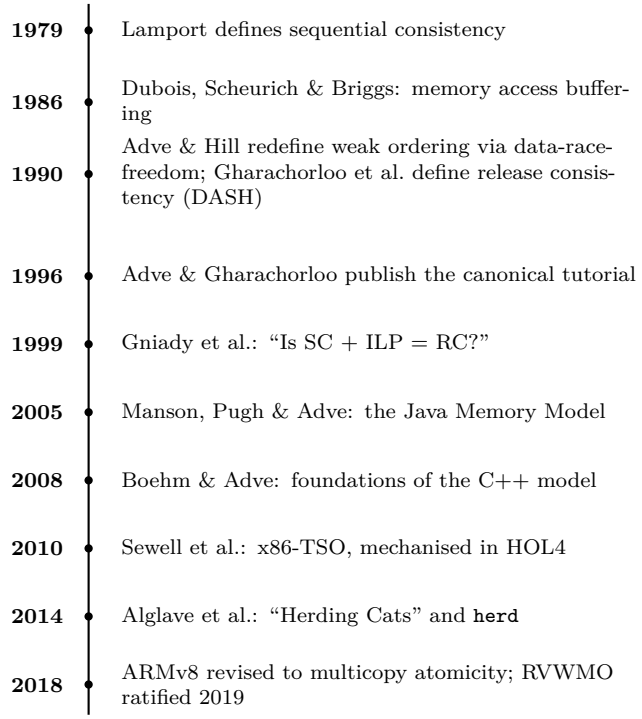

Figure 3: Chronology of memory consistency models.

1979 — the definition. Lamport's paper [19] is two pages and does not argue that sequential consistency is desirable; it assumes so, and asks what a hardware implementation must satisfy. Its significance is that it made the assumption explicit and therefore falsifiable: once SC had a definition, one could build a machine that demonstrably violated it and argue that doing so was a good idea.

1986–1990 — the case for relaxation. Dubois, Scheurich and Briggs [11] observed that SC binds at every memory operation, whereas programs need ordering only where threads communicate. Adve and Hill [3] then inverted the framing: rather than defining weak ordering as a hardware behaviour, they defined it as a contract — write a data-race-free program and the hardware will appear sequentially consistent. This is the most consequential idea in the field. Gharachorloo et al. [12] refined it with release consistency, noting acquire and release each constrain only one direction and so admit cheaper implementations.

The cost of the settlement. The bargain rested on a premise that received less scrutiny than it deserved: that programmers would write data-race-free programs. Java shipped in 1995 with a model that proved simultaneously too weak for the idioms programmers used and too strong for standard compiler optimisations; it was replaced wholesale by JSR-133 [20]. C++ had no memory model at all until C++11 [8]. Alglave et al. [4] formalised the published ARM and POWER manuals and found they did not describe the machines — permitting executions no chip produced and forbidding some that chips did. ARM's 2018 revision to multicopy atomicity was a *strengthening* of a shipped architecture, undertaken because the weaker model was too hard to reason about.

2010 onwards — rigour. Sewell et al. [27] gave x86-TSO in both operational and axiomatic form and proved the two equivalent in HOL4. Alglave et al. [4] unified the axiomatic treatment and supplied executable models. The field is now unusually rigorous for computer architecture, and this rigour is what makes the experimental programme of this paper possible.

2.8 Why This Matters: Three Stakes

The problem is not academic. Adve and Gharachorloo [2] frame the canonical trio:

Correctness
: Synchronisation code — locks, lock-free structures, the internals of language runtimes and OS kernels — is correct only relative to a memory model. The same C code can be correct on x86 and subtly broken on ARM. Sewell et al. [27] verify a Linux spinlock against x86-TSO, and the proof depends on TSO's specific guarantees.

Portability
: Because models differ, a program reasoned about on one architecture may misbehave on another. This is what pushed the problem *up* into programming languages.

Performance
: Every ordering guarantee has a cost. Enforcing sequential consistency naively means a core often cannot proceed past a memory operation until it is globally visible, discarding the very buffering that makes cores fast. Quantifying this trade-off is Experiment B.

2.9 Language-Level Memory Models

Hardware models are not the whole story. Application programmers write in C, C++, Java, Rust and Go, and the compiler is *itself* a source of reordering — it hoists, sinks and eliminates memory accesses. A correct concurrent program therefore needs a contract with the *language*, not just the hardware.

SC-DRF: the unifying idea. The most important idea in language memory models is **SC for data-race-free programs**, whose lineage runs from Adve and Hill [3] through the Java and C++ models. *If a program has no data races — every pair of conflicting accesses is ordered by synchronisation — the programmer is guaranteed sequential consistency, even though hardware and compiler are relaxed.* The bargain: programmers promise to synchronise properly; in return the system promises intuitive semantics. Relaxation is confined to ordinary accesses in correctly synchronised code, where it is invisible, and to explicitly tagged atomics, where the programmer opts into weaker guarantees knowingly.

Java (2005). Java is memory-safe and sandboxed, so it cannot allow racy programs to do *anything* — undefined behaviour would break the security model. The Java Memory Model [20] therefore had the hard job of giving SC-DRF *and* bounding the behaviour of racy programs, via happens-before and causality constraints forbidding values appearing "out of thin air". It is the first industrial-strength language memory model, and the fact that several proposed versions had subtle flaws is itself a lesson in how non-obvious this territory is.

C++11 and `std::atomic`. C++ prioritises performance and control, and exposes the trade-off directly through `std::atomic` operations parameterised by a `memory_order` [8]:

- `memory_order_seq_cst` — the default; participates in a single total order of all seq_cst operations. Strongest and most expensive.
- `memory_order_acquire` / `memory_order_release` — the acquire/release pair from release consistency, giving exactly the ordering MP needs without a full fence.
- `memory_order_acq_rel` — both, for read-modify-write operations.
- `memory_order_relaxed` — atomicity only, no ordering. Cheapest; used for counters and flags where ordering is not required.

This is the cleanest place to *measure* the trade-off empirically, because the same program can be recompiled with different tags and its performance observed. That is precisely the design of Experiment B.

The compiler as a reordering agent. Even on a strongly ordered CPU like x86, a program can exhibit weak-memory bugs because the *compiler* reordered accesses at `-O2`. The language memory model is what makes compiler optimisations legal only up to the point where they remain invisible to correctly synchronised code. Vafeiadis et al. [28] showed that some "obvious" optimisations are unsound under the C11 model — evidence that the language layer is as subtle as the hardware layer, and the reason Experiment B measures *compiled* behaviour rather than reasoning about source.

2.10 Specifying and Verifying Memory Models

Where the preceding subsections show breadth, this one shows depth: how much rigour a single point in the space demands. This is where an introductory course treatment hands off to the research literature.

Two specification styles. Models are given *operationally* — define an idealised machine (cores, store buffers, drain rules) and declare an execution legal iff that machine can produce it — or *axiomatically* — define relations over operations (program order, reads-from, coherence) and declare an execution legal iff those relations satisfy stated acyclicity axioms. The operational form is intuitive and close to hardware; the axiomatic form is compact, comparable across models, and tool-friendly.

A central technical activity is *proving the two coincide* for a given architecture — that the x86-TSO store-buffer machine accepts exactly the executions the x86-TSO axioms allow. Sewell et al. did this in the HOL4 proof assistant [27]. It matters because the operational form guides implementers and the axiomatic form guides tool-builders; a proof that they agree means both communities are discussing the same model.

Litmus testing and `herdtools7`. The empirical backbone of the field is litmus testing: take a small program with a known interesting outcome, run it billions of times on real hardware to see whether the outcome occurs, and compare against what a candidate model predicts. The `herdtools7` suite operationalises this — `litmus7` runs tests on hardware, `herd7` simulates a model — and is the direct descendant of the "herding cats" methodology [4]. This is the tooling Experiment A will use.

The `.cat` language. A significant modern development is that axiomatic models are written in a small domain-specific language, `.cat`, as sets of relation definitions and acyclicity constraints, and then *executed* by `herd7`. A memory model becomes a short, precise, runnable file; the x86-TSO, ARMv8, POWER, RISC-V and C++11 models all exist as `.cat` files. This closes the loop between specification and testing and is why the field is unusually rigorous for computer architecture.

Verification. Beyond testing there is proof: that a microarchitecture or compiler respects a model. This includes microarchitectural consistency verification, compiler-correctness results for the mapping from C++11 atomics to ARM and POWER barriers, and ongoing work on RVWMO and GPU models. A recent survey [1] collects the current state.

2.11 Annotated Literature Survey

The sources below are grouped by role. Annotations state why each matters and where the paper uses it.

Foundational definitions. Lamport [19] defines sequential consistency in two pages; the origin point, and our reference model. Adve and Gharachorloo [2] give the best synthesis of the relaxed-model zoo, organised by the program-order-relaxation and write-atomicity axes; the backbone of our design-space treatment.

Relaxed-model origins. Dubois, Scheurich and Briggs [11] introduced formal relaxation via access buffering and the idea of ordering only at synchronisation. Adve and Hill [3] supplied the programmer-centric reframing (SC-for-DRF) that underlies every

later language model. Gharachorloo et al. [12] introduced release consistency and the acquire/release distinction, realised in Stanford DASH and reused verbatim by C++11.

Language-level models. Manson, Pugh and Adve [20] on the Java Memory Model; Boehm and Adve [8] on the foundations of C++11 atomics; Vafeiadis et al. [28] showing the compiler layer is as subtle as the hardware.

Rigorous hardware models and tooling. Sewell et al. [27] give x86-TSO in operational and axiomatic form, proved equivalent in HOL4 and demonstrated on a real spinlock — our reference model for the hardware experiments. Maranget, Sarkar and Sewell [21] provide the accessible entry to ARM and POWER. Alglave, Maranget and Tautschnig [4] give the unified axiomatic framework, the `herd` simulator and the data-mining methodology that became `herdtools7` — our core experimental tooling.

Reference texts. Nagarajan et al. [23] is the standard modern textbook, open access, with chapters on GPU models and formal tools. Course anchors are Sarangi [26, 25], with Hennessy and Patterson [14] and Kogge [18] for the pipelining roots.

2.12 Open Problems

The subject spans four layers — hardware models, language models, specification styles, and verification — and a term paper can travel a long way horizontally, but the payoff is in choosing a vertical slice and going deep. Problems the paper can point to, or engage with:

1. **Out-of-thin-air values.** Language models still struggle to forbid absurd self-justifying outcomes in relaxed code without also forbidding legitimate compiler optimisations.
2. **Heterogeneous and scoped consistency.** CPU–GPU systems with scoped synchronisation lack a single clean model.
3. **Persistent memory ordering.** Non-volatile memory adds a *durability* order on top of the visibility order.
4. **RVWMO adoption and verification.** A young, open, formally specified model whose hardware conformance is still maturing.
5. **Was the rejection of sequential consistency correct?** SC was rejected on performance grounds around 1990, on machines that did not speculate. Gniady et al. [13] argued that a speculative core — which already has coherence invalidations for detection and a squash path for recovery — can enforce SC cheaply, and later work extended this [9, 7, 22]. No shipping processor implements SC, and the gap is unexplained.

Where our slice sits. The paper's chosen depth is the x86-TSO corner: strong enough to reason about cleanly, rigorously specified, well tooled, and directly connected to the C++11 model the microbenchmarks target. This lets the work be concrete and reproducible while still touching the frontier through the survey.

3 Progress as of 24 August 2026

Summary

WP	Work package	Target	Complete
WP1	Foundations: prescribed textbook	31 Aug	**55%**
WP2	Advanced sources	14 Sep	**30%**
WP3	Literature survey and Zotero	21 Sep	**35%**
WP4	Writing: formal, historical, methodological chapters	24 Aug	**90%**
WP5	Experiment A: `herdtools7` litmus tests	28 Sep	**10%**
WP6	Experiment B: C++11 microbenchmarks + `perf`	19 Oct	**0%**
WP7	Experiment C: gem5 (optional)	02 Nov	**0%**
WP8	Final assembly and appendices	11 Nov	**0%**
	Overall		**28%**

Table 5: Work package status as of 24 August 2026.

Note to the author: every percentage in this section and in the Gantt chart must be corrected to reflect what has *actually* been done before submission. Figures that overstate progress will be contradicted by the next draft and are not worth the marks they might gain.

Gantt Chart

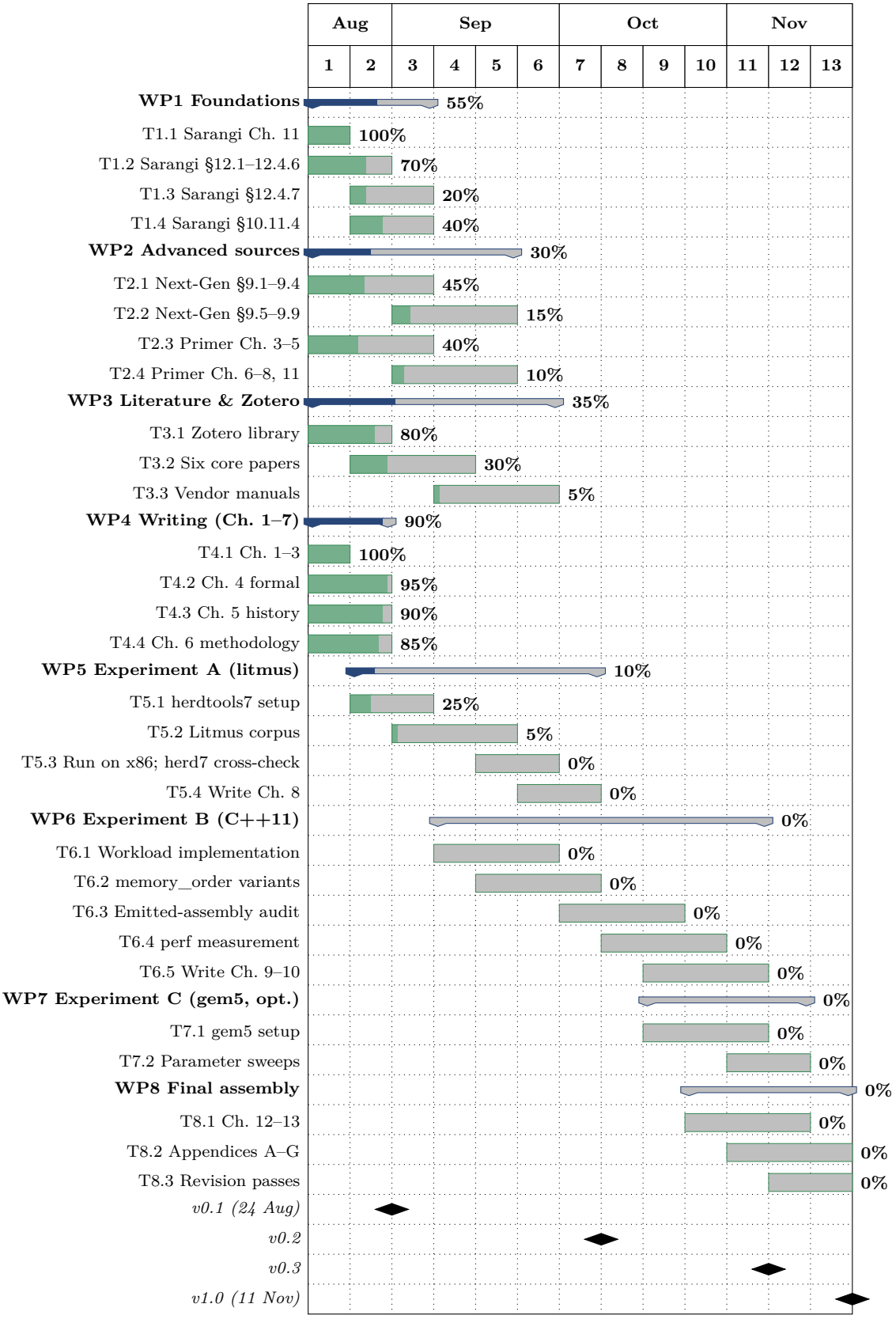

Figure 4: Project schedule. Week 1 commences 17 August 2026; week 13 commences 9 November 2026. Shaded portions of each bar indicate completion. Milestone dates for v0.2 and v0.3 are estimates pending confirmation.

Work Package Detail

WP1 — Foundations (55%).

- **T1.1** Sarangi [26] Ch. 11, *The Memory System*, §11.1–11.3 (pp. 505–560). **100%**. Cache organisation, locality, and the latency figures that motivate the store buffer.
- **T1.2** Sarangi Ch. 12, §12.1–12.4.6 (pp. 565–603). **70%**. §12.4.2 and §12.4.3 read closely; §12.4.6 in progress.
- **T1.3** Sarangi §12.4.7, *Implementing a Memory Consistency Model* (pp. 603–608). **20%**. Begun; requires §12.4.6 first.
- **T1.4** Sarangi §10.11.4, *Out-of-Order Pipelines* (pp. 492–505). **40%**. Needed for the squash mechanism central to Q3.

WP2 — Advanced sources (30%).

- **T2.1** Sarangi [25] §9.1–9.4. **45%**. §9.4 supplies the execution-witness and access-graph notation adopted throughout the paper, chosen over the alternatives for continuity with the course.
- **T2.2** Sarangi §9.5–9.9 (coherence protocols, memory models, race detection, transactional memory). **15%**.
- **T2.3** Nagarajan et al. [23] Ch. 3–5 (SC, TSO, relaxed models). **40%**.
- **T2.4** Nagarajan et al. Ch. 6–8, 11 (coherence protocols, specification and validation). **10%**.

WP3 — Literature and Zotero (35%).

- **T3.1** Zotero library created; 22 entries filed with PDFs. **80%**. Exported to `refs.bib`, under version control.
- **T3.2** Six papers identified as load-bearing for the argument and to be read in full rather than consulted: Lamport [19]; Dubois, Scheurich and Briggs [11]; Adve and Hill [3]; Adve and Gharachorloo [2]; Gniady, Falsafi and Vijaykumar [13]; Alglave, Maranget and Tautschnig [4]. **30%**.
- **T3.3** Vendor manuals [17, 5, 24]. **5%**.

Further references filed and cited: Gharachorloo et al. [12]; Hill [16]; Manson, Pugh and Adve [20]; Boehm and Adve [8]; Ceze et al. [9]; Blundell, Martin and Wenisch [7]; Marino et al. [22]. Supporting texts: Hennessy and Patterson [14]; Culler, Singh and Gupta [10]; Herlihy et al. [15].

WP4 — Writing (90%). A 42-page draft of the paper proper exists in the repository (`main.tex`), comprising:

Ch.	Title	State
1	Introduction — motivating example, problem statement, contributions	drafted
2	Background — memory wall, caches, store buffer, out-of-order	drafted
3	Cache Coherence, and What It Does Not Solve	drafted
4	The Design Space of Memory Consistency Models	drafted
5	Historical Evolution	drafted
6	Methodology	drafted
7	Source Texts and Their Coverage	drafted

Table 6: State of the paper draft. Seven chapters, 42 pages, five TikZ figures, compiling without errors or undefined references.

All figures are drawn in TikZ per the course requirement. The document is built with `latexmk`; `main.pdf` is deliberately excluded from version control and attached instead to tagged releases.

WP5 — Litmus experiments (10%). Design complete and documented in Chapter 6 of the paper draft; implementation begun. The harness must avoid `pthread_barrier`, whose internal fences suppress the window under observation, in favour of a sense-reversing spin barrier built from relaxed atomics. Shared variables are padded to distinct cache lines to prevent false sharing from confounding the coherence behaviour.

WP6–WP8 (0%). Not started; scheduled per the Gantt chart. Designs for WP6 (simulator) and WP7 (verification) are specified in Chapter 6 of the paper draft, stated in advance so that the methodology can be assessed independently of the results.

3.1 Risks and Mitigations

Risk	Sev.	Mitigation
Litmus tests yield zero observations, making Ch. 8 empty	Med	Vary thread placement, optimisation level and inserted delay; a null result with the parameter sweep documented is itself reportable.
Simulator scope creep consumes the schedule	High	Feature list frozen after v0.2. Purpose-built simulator is the baseline; Tejas [25] evaluated in parallel as a stretch goal only, never a dependency.
No AArch64 hardware available for Q2	Med	Single-board computer, Android device under Termux, or institutional server; failing all three, Q2 narrows to x86 and the limitation is stated.
Topic requires material beyond the course	Low	Sarangi's companion volume [25] is pitched at exactly this level and is by the same author.

3.2 Research Questions and Experimental Plan

The remainder of the paper turns from exposition to investigation. The plan is stated now so that the methodology can be assessed in advance of the results.

Research questions

RQ1 Observation

Which "surprising" memory orderings are actually observable on commodity

x86 hardware, and do they match the x86-TSO model exactly? (SB observable; IRIW not; MP safe without a fence.)

RQ2 Cost
What is the concrete performance cost of each C++11 `memory_order` level on the same workload, and how does it map to the fences the compiler emits?

RQ3 Microarchitecture
How do microarchitectural parameters — store-buffer depth, speculation, coherence protocol — change which orderings are observable and how expensive ordering is?

Experiment A — litmus testing on real hardware (primary)

Using `herdtools7` — `litmus7` to run tests on hardware, `herd7` to simulate a model — run the canonical suite (SB, MP, LB, IRIW, plus Dekker-style tests) on an x86 machine, collecting outcome histograms over billions of iterations. Cross-check every observed outcome against the `herd7` prediction under the published x86-TSO `.cat` model. Expected result: SB observable, IRIW never observed, MP safe without barriers, confirming TSO. This answers RQ1.

Adopting `herdtools7` rather than a hand-written harness is a deliberate methodological choice: it is the standard tool of the field, it already implements the affinity and stress options needed to observe rare outcomes, and `herd7` provides the model-checking side without our having to build one.

Experiment B — C++11 atomics microbenchmarks (primary)

Implement small representative workloads — a shared counter, a producer/consumer following the MP pattern, and a spinlock handoff — templated over `memory_order` (relaxed, acquire/release, seq_cst). Compile at a fixed optimisation level, inspect the emitted assembly to record which fences each variant produces, and measure throughput and latency with Linux `perf`. This answers RQ2 and ties source semantics to hardware behaviour to measured cost.

Experiment C — gem5 simulation (optional)

If time allows, use gem5 [6] to vary store-buffer depth and coherence settings and observe the effect on litmus outcomes and microbenchmark cost, isolating causes that are fixed on real silicon. This addresses RQ3. It is flagged optional given its setup complexity; the paper stands on Experiments A and B.

3.3 Immediate Plan to v0.2

1. Complete WP1 and advance WP2; read the six load-bearing papers in full.
2. Install and validate `herdtools7`; reproduce one known litmus result.
3. Run the canonical suite on x86 and record outcome histograms.
4. Implement the three Experiment B workloads and capture emitted assembly.
5. Draft the Experiment A results chapter.
6. Merge the second author's survey draft into the paper proper per `MERGE-PLAN.md`.

3.4 Repository

All work is under version control at
`https://github.com/y4shv/ELL305-term-paper` (private; access granted to the instructor on request).

Draft #1 corresponds to tag `v0.1`. `draft1.tex` builds this progress report; `main.tex` builds the paper draft described in WP4. `refs.bib` is exported from Zotero.

References

[1] Weak memory model formalisms: Introduction and survey. arXiv preprint arXiv:2508.04115, 2025. Verify authors and citation details before final submission.

[2] Sarita V. Adve and Kourosh Gharachorloo. Shared memory consistency models: A tutorial. *IEEE Computer*, 29(12):66–76, 1996.

[3] Sarita V. Adve and Mark D. Hill. Weak ordering — a new definition. In *Proc. 17th International Symposium on Computer Architecture (ISCA)*, pages 2–14, 1990.

[4] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems*, 36(2):7:1–7:74, 2014.

[5] Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture*. Chapter B2: The AArch64 Application Level Memory Model.

[6] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, et al. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, 2011.

[7] Colin Blundell, Milo M. K. Martin, and Thomas F. Wenisch. InvisiFence: Performance-transparent memory ordering in conventional multiprocessors. In *Proc. 36th International Symposium on Computer Architecture (ISCA)*, pages 233–244, 2009.

[8] Hans-J. Boehm and Sarita V. Adve. Foundations of the C++ concurrency memory model. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 68–78, 2008.

[9] Luis Ceze, James Tuck, Pablo Montesinos, and Josep Torrellas. BulkSC: Bulk enforcement of sequential consistency. In *Proc. 34th International Symposium on Computer Architecture (ISCA)*, pages 278–289, 2007.

[10] David E. Culler, Jaswinder Pal Singh, and Anoop Gupta. *Parallel Computer Architecture: A Hardware/Software Approach*. Morgan Kaufmann, 1998.

[11] Michel Dubois, Christoph Scheurich, and Fayé Briggs. Memory access buffering in multiprocessors. In *Proc. 13th International Symposium on Computer Architecture (ISCA)*, pages 434–442, 1986.

[12] Kourosh Gharachorloo, Daniel Lenoski, James Laudon, Phillip Gibbons, Anoop Gupta, and John Hennessy. Memory consistency and event ordering in scalable shared-memory multiprocessors. In *Proc. 17th International Symposium on Computer Architecture (ISCA)*, pages 15–26, 1990.

[13] Chris Gniady, Babak Falsafi, and T. N. Vijaykumar. Is SC + ILP = RC? In *Proc. 26th International Symposium on Computer Architecture (ISCA)*, pages 162–171, 1999.

[14] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.

[15] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.

[16] Mark D. Hill. Multiprocessors should support simple memory consistency models. In *IEEE Computer*, volume 31, pages 28–34, 1998.

[17] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide*. Chapter 9: Memory Ordering.

[18] Peter M. Kogge. *The Architecture of Pipelined Computers*. McGraw-Hill, 1981.

[19] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.

[20] Jeremy Manson, William Pugh, and Sarita V. Adve. The Java memory model. In *Proc. 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 378–391, 2005.

[21] Luc Maranget, Susmit Sarkar, and Peter Sewell. A tutorial introduction to the ARM and POWER relaxed memory models. In *Draft available from the University of Cambridge*, 2012.

[22] Daniel Marino, Abhayendra Singh, Todd Millstein, Madanlal Musuvathi, and Satish Narayanasamy. A case for an SC-preserving compiler. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 199–210, 2011.

[23] Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, and David A. Wood. *A Primer on Memory Consistency and Cache Coherence*. Synthesis Lectures on Computer Architecture. Springer Nature, 2nd edition, 2020.

[24] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Chapter on RVWMO Memory Consistency Model.

[25] Smruti R. Sarangi. *Next-Gen Computer Architecture: Till the End of Silicon*. White Falcon Publishing, 2023. Chapter 9: Multicore Systems — Coherence, Consistency and Transactional Memory.

[26] Smruti R. Sarangi. *Basic Computer Architecture*. White Falcon Publishing, version 3.09 edition, 2025. Available at `https://srsarangi.github.io/archbook/`.

[27] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. x86-TSO: A rigorous and usable programmer's model for x86 multiprocessors. *Communications of the ACM*, 53(7):89–97, 2010.

[28] Viktor Vafeiadis, Thibaut Balabonski, Soham Chakraborty, Robin Morisset, and Francesco Zappa Nardelli. Common compiler optimisations are invalid in the C11 memory model and what we can do about it. In *Proc. 42nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 209–220, 2015.